Cheatsheet Frontend Angular

Reglas de oro

- **El backend valida todo.** Guards y botones ocultos son experiencia de usuario, no seguridad.
- **Nada secreto en un environment**: termina en el bundle y se lee con las DevTools.
- `npm ci` al clonar y en CI. `npm install` solo para cambiar dependencias (toca el lock).
- Node de `.nvmrc` (`nvm use`). Otra versión = "en mi máquina sí funciona".
- Estado **inmutable**: arreglos y objetos nuevos, nunca `push` ni `obj.x = …`.
- Nunca tragues errores: `catchError(() => EMPTY)` sin aviso deja spinners eternos.
- Rutas siempre **lazy**; privadas siempre con `canMatch`.

Arranque

```
nvm use  # Node 22
cd practica/katas && npm ci && npm test  # rojo
cd ../dojo-app && npm ci && npm test  # verde
npm run start:mock  # sin backend, :4200
npm start  # dojo-api en :1099
```

Token local del dojo-api y del mock: `dojo-local-inseguro`. Swagger: `localhost:1099/swagger/index.html`.

Comandos del dojo

Comando	Para qué
`npm run test:typescript` / `test:rxjs`	Katas 02 y 03 (katas/)
`npm run test:solucion`	Solución de referencia: `./ejercicios` se mapea a `./solucion`
`npm test`	Base de dojo-app (sin retos)
`npm run reto:a` · `b` · `c`	Retos de los módulos 06, 09 y 14
`npm run lint` / `format:check`	angular-eslint / Prettier
`npm run build`	Producción en `dist/dojo-app/browser`
`/progreso` `/pista` `/revisar-reto` `/examen`	Sensei en Claude Code

Estructura de un front empresarial

```
src/app/
  core/  # 1 instancia: auth, http, errores, ui, mock
  shared/  # reutilizable sin estado: componentes, pipes
  features/  # 1 carpeta por módulo de negocio
    almacenes/
      models/ services/ state/ pages/
      almacenes.routes.ts
src/environments/  src/testing/
```

Alias: `@core/*`, `@shared/*`, `@env/*`. Nada de `../../../`.

TypeScript del día a día

`Respuesta<T>`	Genérico: una envoltura para todo
`'todos' | 'activos'`	Unión de literales en vez de strings sueltos
`Partial / Pick / Omit`	Derivar tipos, no copiarlos
`Record<K, V>`	Diccionarios
`readonly T[]`	Que nadie mute
`a?.b ?? 'x'`	Encadenamiento opcional + valor por defecto
`unknown` vs `any`	unknown obliga a validar; any apaga el compilador

```
type Estado<T> =
  | { tipo: 'cargando' }
  | { tipo: 'exito'; datos: T }
  | { tipo: 'error'; mensaje: string };
switch (e.tipo) { …
  default: { const n: never = e; return n; } }
function esAlmacen(x: unknown): x is Almacen {
  return typeof x === 'object' && x !== null && …;
}
```

`http.get<T>()` no valida nada: el tipo es una promesa al compilador.

RxJS: operadores clave

`map`	Desenvolver `r => r.datos`
`filter`	Ignorar (diálogo cancelado)
`tap`	Efecto lateral sin cambiar el valor
`catchError`	Convertir o re-lanzar normalizado
`finalize`	Apagar carga: corre en éxito, error y cancelación
`debounceTime` + `distinctUntilChanged`	Buscadores
`combineLatest`	Lista + filtros
`startWith`	Estado inicial "cargando"
`retry(n)`	Reintentar (solo idempotentes)
`shareReplay`	Caché de catálogos (`bufferSize: 1`)

Los cuatro *Map

Operador	Si llega otro valor…	Úsalo en
`switchMap`	cancela el anterior	buscador, cambio de selección
`concatMap`	encola	guardados en orden
`mergeMap`	en paralelo	cargas independientes
`exhaustMap`	lo ignora	botón Guardar con doble clic

```
texto$.pipe(debounceTime(300), map(t => t.trim()),
  distinctUntilChanged(),
  switchMap(t => buscar(t).pipe(
    catchError(() => of([])))));
```

Un Observable que falla **muere**: atrapa el error **dentro** del switchMap.

Los cuatro *Map

Suscripciones (mejor a peor)

`toSignal()` / pipe `async` · `takeUntilDestroyed()` · `take(1)` · `unsubscribe()` en `ngOnDestroy`.

Canicas

`-` 1 ms · `a` valor · `|` completa · `#` error · `(ab)` mismo ms. `TestScheduler.run` usa tiempo virtual.

Componente standalone

```
@Component({
  selector: 'app-page-header',
  imports: [MatButtonModule],  // lo que usa la plantilla
  changeDetection: ChangeDetectionStrategy.OnPush,
  templateUrl: './page-header.component.html',
})
export class PageHeaderComponent {
  readonly titulo = input.required<string>();
  readonly subtitulo = input<string>();
  readonly cerrar = output<void>();
  private readonly store = inject(AlmacenesStore);
}
```

`model()` = input escribible (`[(valor)]`). `<ng-content />` = hueco para contenido.

Plantillas

```
{{ titulo() }}  [disabled]="guardando()"
(click)="guardar()"
@if (error(); as e) { <p role="alert">{{ e }}</p> }
@else { … }
@for (p of perfiles; track p.clave) { … }
@empty { Sin datos }
@switch (vista().tipo) { @case ('ok') { … }
  @default { … } }
@defer (on viewport) { <app-grafica /> }
@placeholder { … }
```

`track` con id estable, nunca `$index` si la lista se reordena. Pipes puros mejor que métodos en la plantilla.

Signals

`signal(v)`	Estado que cambia: `.set()` `.update(fn)`
`computed(() => …)`	Derivado; solo se recalcula si cambian sus dependencias
`effect(() => …)`	Efecto lateral. Úsalo poco; no escribas signals dentro
`linkedSignal`	Escribible que se reinicia cuando cambia otro
`toSignal(obs$)`	Observable a signal (se desuscribe solo)
`toObservable(sig)`	Signal a Observable
`.asReadonly()`	Exponer sin permitir `.set()`

DI y detección de cambios

- `@Injectable({ providedIn: 'root' })`: singleton y tree-shakable.

DI y detección de cambios

- `InjectionToken` para valores (`API_URL`); en tests: `{ provide: API_URL, useValue: '/api' }`.
- `inject()` solo en contexto de inyección: campos, constructor, guards, interceptores.

zone.js	Revisa tras cada evento, timer o petición. La mayoría de los proyectos existentes.
OnPush	Solo si cambia un input (por referencia), un evento propio o un signal leído.
Zoneless	Default en apps nuevas de Angular 21. Signals + eventos.

> "No se actualiza hasta que paso el mouse": mutación o cambio async sin signal en OnPush.

Rutas y lazy loading

```
{ path: 'almacenes',
  canMatch: [authGuard, permisoGuard('almacenes.ver')],
  loadChildren: () =>
    import('./features/almacenes/almacenes.routes')
      .then(m => m.ALMACENES_ROUTES) },
{ path: 'salud',
  loadComponent: () =>
    import(…).then(m => m.SaludComponent) },
{ path: '**', loadComponent: … }  // siempre al final
```

`provideRouter(routes, withHashLocation(), withComponentInputBinding())`: `:id` llega como `input()`.

Hash `/#/x`	Cualquier hosting estático, sin reglas en el servidor
Path `/x`	URLs limpias; el servidor reescribe todo a `index.html`

Guards funcionales

`canMatch`	Sesión y permisos. Si falla, el chunk lazy **no se descarga**
`canActivate`	Ya elegida la ruta
`canDeactivate`	Cambios sin guardar
`resolve`	Precargar (bloquea la navegación)

```
export const authGuard: CanMatchFn = (_r, segs) =>
  inject(SesionService).autenticado()
    ? true
    : inject(Router).createUrlTree(['/login'],
        { queryParams: { volver } });
```

- Devuelve `UrlTree`, no navegues dentro del guard.
- Observable en guard: `take(1)` (canMatch corre en cada navegación).
- `canLoad` está obsoleto: es `canMatch`.

Formularios reactivos tipados

```
form = inject(NonNullableFormBuilder).group({
  clave:  ['', [Validators.required, claveValida]],
  nombre: ['', [Validators.required, noSoloEspacios,
                Validators.maxLength(80)]],
});
guardar() {
  if (this.form.invalid) {
    this.form.markAllAsTouched(); return;
  }
  const { clave, nombre } = this.form.getRawValue();
  // { clave: string; nombre: string }
}
```

- `getRawValue()` incluye deshabilitados; `value` no.
- `required` acepta `" "`: usa `noSoloEspacios`.
- No deshabilites Guardar por invalidez; sí mientras guarda.
- 409 del servidor junto al campo: `control.setErrors({ servidor: msg })`.
- Validador = `(control) => ValidationErrors | null`, una regla y un nombre.

Servicio HTTP tipado

```
@Injectable({ providedIn: 'root' })
export class AlmacenesService {
  private http = inject(HttpClient);
  private url = `${inject(API_URL)}/almacenes`;
  listar(soloActivos = false): Observable<Almacen[]> {
    const params = soloActivos
      ? new HttpParams().set('activos', 'true')
      : undefined;
    return this.http
      .get<Respuesta<Almacen[]>>(this.url, { params })
      .pipe(map(r => r.datos ?? []));
  }
}
```

Devuelve Observable · desenvuelve `datos` · sin toasts · `HttpParams`, no strings concatenados.

Interceptores y política de errores

```
withInterceptors([carga, auth, error,
  ...(mock ? [mockInterceptor] : [])])
petición:  carga -> auth -> error -> mock/backend
respuesta: carga <- auth <- error <- mock/backend
```

Status	Quién lo muestra
0, 403, 5xx	Interceptor (toast)
401	Interceptor: cierra sesión y va a /login
400, 404, 409	Componente, junto al campo o la tabla

- `req.clone({ setHeaders })`: las peticiones son inmutables.
- Token solo a **tu** API (`req.url.startsWith(apiUrl)`).
- Opciones por petición: `HttpContextToken` (`silencioso()`), no headers inventados.
- `ErrorApp.notificado` evita mostrar dos veces.

Proxy, CORS y mock

```
// proxy.conf.json
{ "/api": { "target": "http://localhost:1099",
            "pathRewrite": { "^/api": "" } } }
```

`apiUrl: '/api'` en todos los environments: sin CORS en desarrollo y sin dominios en el bundle. En producción, un reverse proxy.

> El mock responde con el **mismo contrato** (envoltura, códigos 401/404/409). Si cambia el backend, cambia el mock en el mismo PR.

Store con signals

```
private readonly _items = signal<Almacen[]>([]);
readonly items = this._items.asReadonly();
readonly filtro = signal('');
readonly filtrados = computed(() =>
  filtrar(this._items(), this.filtro()));
crear(a: AlmacenAlta) {
  return this.service.crear(a).pipe(
    // pesimista: después de que el backend confirma
    tap(n => this._items.update(l => [...l, n])));
}
```

Local	`signal` en el componente
Compartible por URL	Query params (`?q=`, página)
De feature	Store de la feature
Global	Servicios de `core/` (sesión, carga)

NgRx Signal Store: muchas features CRUD iguales. NgRx Store: estado global complejo y muchos equipos.

Angular Material

`mat-table` + `*matNoDataRow`	Tabla con estado vacío
`mat-paginator`	Al filtrar, regresa a la página 1
`mat-form-field` + `mat-error`	Campos con error
`MatDialog`	`afterClosed()` -> Observable
`MatSnackBar`	Un servicio único de avisos
`MatPaginatorIntl`	Traducir textos de Material

```
html { @include mat.theme((color: mat.$azure-palette)); }
.x { color: var(--mat-sys-on-primary-container); }
```

> Nada de colores fijos ni `::ng-deep`: usa variables `--mat-sys-*`.

Accesibilidad y formato es-MX

- Todo control con nombre: `mat-label`, `aria-label="Editar ALM-01"`.
- `<button>` para acciones, `<a routerLink>` para navegar.
- `role="alert"` en errores, `role="status"` en avisos.
- Si `getByRole` no lo encuentra, un lector de pantalla tampoco.

```
registerLocaleData(localeEsMx);
{ provide: LOCALE_ID, useValue: 'es-MX' }
{{ total | currency: 'MXN' }}  $1,234.50
{{ fecha | date: 'mediumDate' }}  1 nov 2026
```

Testing con Jest

```
npx jest almacenes  # por archivo
npx jest -t "filtra"  # por nombre
npm run test:coverage
```

HttpTestingController

```
providers: [provideHttpClient(),
            provideHttpClientTesting()]
service.listar().subscribe(l => (r = l));
backend.expectOne('/api/almacenes')
  .flush({ codigo: 200, mensaje: 'ok', datos: [] });
afterEach(() => backend.verify());
```

Guards

```
TestBed.runInInjectionContext(() =>
  authGuard(ruta, segmentos));
```

Testing Library y Playwright

```
await render(ListaComponent,
  { providers: [...proveerApiMock()] });
const caja = screen.getByRole('searchbox',
  { name: 'Buscar' });
await userEvent.type(caja, 'bodega');
expect(screen.getByText('1 – 2 de 2'))
  .toBeInTheDocument();
```

`getBy`	Falla si no está
`queryBy`	`null` si no está (afirmar ausencia)
`findBy`	Espera (asíncrono)

Prioridad: rol + nombre > label > texto. Nunca clases CSS.

```
npx playwright install chromium && npm run e2e
```

E2E: pocos, de los flujos que más importan.

Build y environments

```
"development": { "fileReplacements": [{
  "replace": "src/environments/environment.ts",
  "with":
    "src/environments/environment.development.ts" }] }
```

- Un environment por build; forma común en `environment.model.ts`.
- **Initial total** = lo que se descarga antes de pintar; lo vigilan los budgets.
- Archivos con hash = caché larga. `index.html` sin caché.
- Un solo build para varios entornos: `config.json` + `provideAppInitializer`.

CI mínimo de un front

```
- uses: actions/setup-node@v4
  with: { node-version-file: .nvmrc, cache: npm }
- run: npm ci
- run: npm run lint
- run: npm run format:check
- run: npm test -- --ci
- run: npm run build
```

Mensajes de commit validados con commitlint + husky en muchos equipos: lee el error, casi siempre es el formato.

Seguridad en el front

`{{ x }}`	Escapado: seguro
`[innerHTML]`	Sanitizado por Angular
`bypassSecurityTrust*`	Peligro con datos del usuario
`el.innerHTML = …`	Te saltas a Angular: XSS

- Token: memoria > `sessionStorage` > `localStorage`. Mejor: cookie `HttpOnly`.
- Nunca token en query params, logs ni mensajes de error.
- `?volver=`: solo rutas internas (no `//`, `/\`, `https:`).
- Sin source maps ni detalles técnicos del backend en producción.
- `npm audit --omit=dev`. Cabeceras (CSP, HSTS) las pone el servidor.

Lo vas a encontrar así

Legado	Moderno
`NgModule` + `SharedModule`	Standalone + `imports`
`constructor(private s: X)`	`inject(X)`
`*ngIf` `*ngFor`	`@if` `@for … track`
`@Input()` `@Output()` `ngOnChanges`	`input()` `output()` `computed`
`BehaviorSubject` + `async`	`signal` + `computed`
`UntypedFormGroup`	`NonNullableFormBuilder`
Guards de clase, `canLoad`	`CanMatchFn`
`HTTP_INTERCEPTORS` de clase	`withInterceptors([fn])`
`.toPromise()`	`firstValueFrom`
Bootstrap grid, `::ng-deep`	CSS grid/flex, variables del tema
`moment`	pipe `date`, `Intl`

```
ng generate @angular/core:control-flow
ng generate @angular/core:signal-input-migration
ng generate @angular/core:inject
```

Checklist antes de abrir un PR

- `npm test`, `npm run lint`, `npm run format:check`, `npm run build` en verde.
- Sin `any`, sin `console.log`, sin código comentado.
- Rutas nuevas lazy y con `canMatch`; permiso registrado.
- Errores: nada tragado, 4xx junto al campo, spinners que se apagan.
- Estado inmutable; nada de suscripciones colgando.
- Botones con nombre accesible; probado con teclado.
- Mock actualizado si cambió el contrato.
- Descripción: qué, por qué, cómo se probó, capturas.